CODEGNAN-HACKATHON

Data Visionaries

Customer Support Ticketing System with SLA Tracking

Domain: Customer Support

1. Problem Statement

Support teams handling customer issues over email or chat often lose track of which tickets are overdue, leading to missed service-level agreements (SLAs) and frustrated customers. There is no structured way to prioritize, assign and escalate tickets automatically.

2. Goal

Build a support ticketing system where customers raise issues, tickets are automatically assigned and tracked against SLA deadlines, and support managers get visibility into overdue tickets and team performance.

3. Input / Requirements

A. User Roles

- Customer
- Support Agent
- Support Manager / Admin

B. Registration & Login

Customers register to raise tickets; agents and managers have internal role-based accounts.

C. Data Entry

- Ticket: subject, description, category, priority, customer
- SLA rule: priority level, response/resolution time limit
- Agent response/resolution notes

D. Core Functional Requirements

- Customer raises a ticket with category and priority
- System auto-assigns ticket to an available agent or queue
- SLA timer starts based on priority
- Agent responds/resolves, updating status
- Manager dashboard for overdue tickets and agent workload
- Escalation flag for tickets nearing/breaching SLA

E. Business Workflow

Customer raises a ticket → Ticket is assigned to an agent → SLA clock starts based on priority level → Agent works the ticket and updates status → System flags tickets nearing SLA breach → Manager reviews overdue tickets and reassigns if needed → Ticket is resolved and customer can rate the resolution

F. Database Requirements (key entities)

- Customers
- Tickets
- Agents
- SLARules
- TicketStatusHistory

G. Search / Filter / Analytics

- SLA compliance rate
- Average resolution time by category
- Agent workload and performance ranking

H. Advanced / Optional Features

- Automated SLA breach alerts
- Simple rule-based auto-assignment (round robin or category-based routing)

4. Suggested Tech Stack

Frontend	React / Angular
Backend	Java Spring Boot
Database	MySQL / PostgreSQL
Data / ML	Not core to this problem
Visualization	Chart.js for SLA dashboard
Deployment	Any PaaS

Students are free to choose an alternative technology stack based on their team's expertise, provided they can demonstrate the complete working solution.

5. Output / Expected Deliverable

A working ticketing system with SLA-aware assignment, breach alerts, and a manager dashboard tracking SLA compliance and agent performance.

Expected Modules / Screens

- Ticket creation & assignment
- SLA tracking engine
- Agent workspace
- Manager SLA dashboard

Final Demo Flow

1. Customer raises a ticket
2. System assigns it to an agent and starts the SLA timer
3. Agent responds and updates ticket status

4. System flags a different ticket nearing SLA breach
5. Manager dashboard shows SLA compliance rate for the day

Hackathon Expectations

By the end of the hackathon, the team should be able to demonstrate:

- A working, demonstrable application — not just screens or static mockups
- A clean, usable interface appropriate for the target user
- A functional backend with real business logic, not placeholder responses
- A connected database with meaningful, non-trivial data relationships
- Input validation and sensible error handling
- Authentication and role-based access control where applicable
- Basic testing of core flows before the demo
- Source code shared with the evaluation team
- A short README describing setup steps and key design decisions
- A live demo walkthrough covering the flow described above